

个人作业报告

题目：**编译原理**

实验课第四次作业

姓名	林宏宇
学号	23320093
院系	计算机学院
专业	计算机科学与技术
指导教师	李文军

2026 年 5 月 18 日

【摘　要】

根据实验内容[1],在此次实验需要实现一个基于 Java 的表达式计算器，此报告参照评分标准[2]，完成这份实验报告撰写并附上相应的实验报告说明。

本实验设计并实现了完整的编译器前端系统，包括：（1）词法分析器，能够识别 Pascal 风格数值常量（含科学记数法）、布尔常量、预定义函数名与各类运算符，并对非法词法单元进行精确错误检测；（2）基于 11 级优先级的算符优先语法分析器，正确处理一元取负与二元减法的重载消歧义、幂运算和三元运算的右结合性、预定义函数的可变参数；（3）严格的语义处理程序，通过类型推导机制实现类型兼容性检查，确保布尔值不参与算术运算、数值值不参与逻辑运算。本实验共编写了 49 个自定义扩展测试用例，综合覆盖 simple.xml、standard.xml 和 mytest.xml 三套测试集全部 73 个用例，通过率均为 100%，并生成了完整的 javadoc 文档。此报告详细阐述了二义性分析、词法分析、优先级表构造、语法分析与语义处理的设计与实现，并报告了实验结果与工程实践成果。

关键词：　编译原理；表达式计算

目录

1 实验目的 1

2 项目组织 2

3 实验实现 3
3.1 语法定义的二义性分析 3
3.2 词法分析程序的设计与实现 4
3.3 算符优先关系表的构造 5
3.4 语法分析与语义处理程序的设计与实现 7
3.5 自定义测试用例设计与覆盖 9
3.6 异常处理 10
3.7 有限状态自动机图表 12

4 实验结果与分析 13
4.1 编译结果 13
4.2 GUI 界面基本功能 14
4.3 自动回归测试结果分析 16
4.4 手动计算测试 19

5 自我评分参照 20

6 实验总结 21
6.1 技术总结 21
6.2 心得体会 21

参考文献 23

附录 A 附录代码 24
A.1 Calculator.java 代码 24

1 实验目的

本实验旨在设计并实现一个基于表达式的计算器 **ExprEval**，通过完整的词法分析、算符优先语法分析（Operator Precedence Parsing, OPP）和语义处理，深入理解和掌握编译原理中的核心技术。主要目标包括：

- **词法分析程序的设计与实现**：根据表达式语言的词法规则，构造有限状态自动机作为程序蓝图，识别 Pascal 风格的数值常量（含科学记数法）、布尔常量、预定义函数名、各类运算符等，并对非法词法单元进行精确的错误检测与分类。
- **算符优先分析技术**：构造包含 11 级优先级的算符优先关系表，处理重载的一元/二元运算符（如一元取负与二元减法）、三元运算符（?:）、预定义函数等，正确处理左结合与右结合性质。
- **语义处理技术**：完成类型推导与类型兼容性检查，正确区分数值类型和布尔类型的子表达式，检测除数为零等运行时语义错误。
- **面向对象设计与软件工程实践**：采用面向对象风格设计类结构，编写 javadoc 文档化注释，编写自定义测试用例并通过实验软装置完成回归测试。

2 项目组织

- **src/parser/**：源代码目录。
  - `Calculator.java`：表达式计算器的核心实现，包含词法分析器（Lexer）、算符优先语法分析器（Parser）和语义处理程序（Evaluator）的全部代码。
- **bin/**：编译生成的字节码目录。
  - `bin/parser/`：Calculator 编译后的 class 文件。
  - `bin/exceptions/`：实验软装置预定义的 16 种异常类。
  - `bin/gui/`：GUI 主窗口及其内部类（预编译）。
  - `bin/test/`：自动回归测试程序（预编译）。
- **doc/**：javadoc 生成的 HTML 格式设计文档。
- **testcases/**：测试用例 XML 文档目录。
  - `simple.xml`：简单测试用例（8 个）。
  - `standard.xml`：标准测试用例（16 个，覆盖所有简单用例）。
  - `mytest.xml`：自定义扩展测试用例（49 个，覆盖边缘情况）。
- **build.bat / Makefile**：Windows batch 脚本和跨平台 Makefile，支持构建、测试、运行与文档生成。

3 实验实现

3.1 语法定义的二义性分析

根据实验规格说明中给出的 BNF 定义（第 2.3.1 节），ExprEval 所支持的表达式语言存在二义性。

二义性来源分析：

1) **一元取负与二元减法运算符的重载**：BNF 中一元取负 – ArithExpr 与二元减法 ArithExpr – ArithExpr 使用了相同的终结符 "–"。对于输入 "2 – 3 ∗ –4"，减号既可以被解释为一元取负（–4），也可以被解释为二元减法（2 – 3 与某个表达式的差）。这种重载导致语法分析器需要通过上下文来判定减号的真实含义。
2) **三元运算符 ? : 的嵌套**：BNF 中 BoolExpr ? ArithExpr : ArithExpr 是一个右结合的产生式。对于 "$a\,?\,b\,?\,c\ :\ d\ :\ e$"，如果不规定结合性，既可以理解为 $a\,?\,(b\,?\,c\ :\ d)\ :\ e$（右结合），也可以理解为 $(a\,?\,b\,?\,c\ :\ d)\ :\ e$（错误解析）。若语法未明确结合性质，则存在二义性。
3) **布尔表达式与算术表达式的上下文依赖**：BoolExpr 和 ArithExpr 在 BNF 中相互引用。例如 BoolExpr 可推导出 ArithExpr > ArithExpr，而 ArithExpr 又可包含 BoolExpr ? ArithExpr : ArithExpr。这种交叉引用在纯 BNF 层面构成间接二义性，因为比较运算的结果（布尔类型）在语法层面并未与算术运算的输入（数值类型）作出区分，需要借助语义分析中的类型推导来分辨合法与非法表达式。

二义性的消除：

本实验通过算符优先分析技术消除二义性：

- **一元取负与二元减法**：通过上下文状态标志 `expectOpnd` 来区分。当语法分析器期望一个运算量时，遇到的 "–" 识别为一元取负；当期望一个运算符时，遇到的 "–" 识别为二元减法。一元取负具有比二元减法（以及幂运算）更高的优先级（级别 9），确保了 "-3^2" 被解析为 $(-3)^2$ 而非 $-(3^2)$。
- **三元运算符嵌套**：明确规定 ? : 为右结合，并在算符优先关系表中通过右结合的处理规则来实现：当栈顶运算符与当前 ? 优先级相同时，不归约栈顶，而是将新的 ? 压栈，从而保证右侧的 ? : 优先配对。
- **类型二义性**：在语义处理阶段通过类型检查机制解决。本实验使用 `Val` 内部

类记录了每个值的类型（`isBool` 字段），在归约时检查操作数类型与运算符的兼容性，对不兼容的组合抛出 `TypeMismatchedException`。

3.2 词法分析程序的设计与实现

3.2.1 词法规则

ExprEval 所支持表达式语言的词法规则如下：

- **空白符号**：仅使用空格（Space）作为单词分隔符，多个连续空格视为单个分隔。制表符、回车、换行、注释等均不允许出现。
- **数值类型常量**：支持 Pascal 风格的无符号数值常量，符合以下正则定义：

$$
\begin{aligned}
\mathit{digit} &\rightarrow 0|1|2|3|4|5|6|7|8|9 \\
\mathit{integral} &\rightarrow \mathit{digit}^{+} \\
\mathit{fraction} &\rightarrow . \ \mathit{integral} \\
\mathit{exponent} &\rightarrow (E \mid e)\ (+ \mid - \mid \varepsilon)\ \mathit{integral} \\
\mathit{decimal} &\rightarrow \mathit{integral}\ (\mathit{fraction} \mid \varepsilon)\ (\mathit{exponent} \mid \varepsilon)
\end{aligned}
$$

  注意：不支持负数常量（如 −3.14 是数值常量 3.14 的一元取负运算），小数点前后必须有数字。
- **布尔类型常量**：仅允许 `true` 和 `false`，大小写无关。
- **预定义函数名**：`sin`、`cos`、`max`、`min`，大小写无关。
- **运算符**：包括算术运算符（`+ - * / ^`）、关系运算符（`= <> < <= > >=`）、逻辑运算符（`& | !`）、三元运算符（`? :`）、括号和逗号。

3.2.2 词法分析实现

词法分析程序嵌入在 `Calculator` 类中，核心方法为 `lex()`。该方法逐字符扫描输入字符串，根据首字符将不同类别单词的识别分派给三个私有方法：

- `lexNumber()`：识别数值常量。按照上述正则定义构造识别流程：先读取整数部分（`integral`），若遇小数点则读取小数部分（`fraction`），若遇 `e/E` 则读取指数部分（`exponent`）。每一步若发现格式错误（如小数点后无数字、`E` 后无指数等），立即抛出 `IllegalDecimalException`。特别注意：以 “.” 开头的数值（如 `.11`）缺少整数部分，直接抛出异常；数值后若紧跟字母或额外

小数点（如 12.3Emax、3.3e3.3），同样判定为非法数值常量。

- lexIdent()：识别标识符。读取连续字母序列后，在关键字表中匹配。若为 true 或 false（大小写无关），生成布尔类型 Token；若为 sin、cos、max 或 min，生成函数类型 Token；否则抛出 IllegalIdentifierException。例如，mix 导致词法错误，而 sin5 被正确地识别为函数名 sin 加上数值常量 5（因标识符与后续单词之间无需空白分隔）。
- lexSymbol()：识别运算符和界符。单字符运算符（+ * / ^ & | ! ( ) , ? : - =）直接匹配；双字符运算符（<= >= <>）在读取首字符后前瞻（lookahead）下一个字符确认。

3.3 算符优先关系表的构造

3.3.1 优先级定义

本实验按照规格说明定义了严格的 11 级优先级体系，如表 3.1 所示。优先级数值越高表示结合越紧密。

级别（高 → 低）	算符	描述	结合性质
1（最紧）	( )	括号	——
2	sin cos max min	预定义函数	——
3	-（一元）	取负运算	右结合
4	^	求幂运算	右结合
5	* /	乘除运算	左结合
6	+ -	加减运算	左结合
7	= <> < <= > >=	关系运算	左结合
8	!	非运算	右结合
9	&	与运算	左结合
10	|	或运算	左结合
11（最松）	?:	选择运算	右结合

表 3.1 算符优先级与结合性质表

3.3.2 优先级函数实现

优先级由 prec() 函数实现，该函数根据具体 Token 类型和是否为一元运算符返回整型优先级值：

```
1 private static int prec(TokenType t, boolean unary) {
2     if (unary && t == TokenType.MINUS) return 9;
3     switch (t) {
4         case QUEST: case COLON: return 1;
```

```
        case OR:                return 2;
        case AND:               return 3;
        case NOT:               return 4;
        case EQ: case NE:
        case LT: case LE:
        case GT: case GE:       return 5;
        case PLUS: case MINUS: return 6;
        case MUL: case DIV:     return 7;
        case POWER:             return 8;
        default:                return 0;
    }
}
```

关键设计点：

- 一元取负（`unary==true && t==MINUS`）返回优先级 9，高于幂运算（8），确保 "-3^2" 解析为 $(-3)^2 = 9$。
- 三元运算符 ? 和 : 处于最低优先级 1，使得关系运算和逻辑运算的结果优先于三元选择被归约。
- 括号和函数调用不参与优先级比较，而是在算符栈中作为特殊的同步标记（marker）处理。

3.3.3 结合性质处理

右结合性由 `rightAssoc()` 函数判定，^ 和 ? 为右结合。在 `evalBeforePush()` 中，对于右结合运算符，仅当栈顶优先级**严格大于**当前优先级时才归约（$pTop > pIn$），而非左结合时的 $pTop \geq pIn$：

```
if (rightAssoc(incoming.type)) {
    if (pTop <= pIn) break;   // 右结合：栈顶优先级 <= 当前则停止归约
} else {
    if (pTop < pIn) break;    // 左结合：栈顶优先级 <  当前则停止归约
}
```

这一设计保证了对 "2 ^ 4 ^ 3" 的正确解析：第二个 ^ 入栈时，栈顶的 ^ 优先级等于当前，因右结合特性停止归约，最终归约顺序为 2 ^ (4 ^ 3)。

3.4 语法分析与语义处理程序的设计与实现

3.4.1 算符优先分析核心算法

语法分析程序采用双栈结构的算符优先分析方法。两个核心栈分别为：

- `valStack`（`Stack<Val>`）：存放运算量（数值或布尔值）。
- `opStack`（`Stack<OpEntry>`）：存放运算符和界符。`OpEntry` 封装了 Token、是否为一元运算符标记、以及函数调用的参数基址。

此外，三元运算符 "?:" 的实现使用了辅助栈 `ternStack`（`Stack<TernState>`），用于在 "?" 和 ":" 被分别归约时暂存条件和真值分支。

核心解析循环在主方法 `calculate()` 中实现。对每个 Token 进行分类处理后，使用 `expectOpnd` 布尔标志跟踪期望的下一个单词类型（运算量或运算符），以此精确检测 `MissingOperandException` 和 `MissingOperatorException`。

归约动作的核心在 `evalOp()` 中完成，其对各运算符的语义处理包括类型检查和计算：

```
private void evalOp(OpEntry oe) throws ExpressionException {
    Token op = oe.tok;

    // 三元 ':' 归约
    if (op.type == TokenType.COLON) {
        Val falseVal = valStack.pop();
        TernState ts = ternStack.pop();
        if (!ts.cond.isBool) throw new TypeMismatchedException();
        Val res = (ts.cond.v != 0) ? ts.trueBranch : falseVal;
        valStack.push(res);
        return;
    }
    // 一元取负
    if (op.type == TokenType.MINUS && oe.isUnary) { ... }
    // 一元逻辑非
    if (op.type == TokenType.NOT) { ... }
    // 二元运算：弹出左右操作数，类型检查，计算并压栈
    Val b = valStack.pop();
    Val a = valStack.pop();
    switch (op.type) { ... }
}
```

3.4.2 重载运算符的处理

表达式语言中存在重载的运算符“-”：它既可以是一元取负，也可以是二元减法。本实验通过 expectOpnd 标志进行消歧义：

```
case MINUS:
    if (expectOpnd) {
        opStack.push(new OpEntry(t, true));  // 一元取负
        sawComma = false;
    } else {
        evalBeforePush(t, false);             // 二元减法：先归约栈中更高优先级的运算
        opStack.push(new OpEntry(t, false));
        expectOpnd = true;
    }
    pos++;
    break;
```

当期望一个运算量时，遇到的“-”被识别为取负运算符并压入 opStack（标记为 isUnary=true）；当期望运算符时，遇到的“-”按二元减法处理，先归约栈顶所有优先级不低于它的运算符再入栈。

3.4.3 类型推导与兼容性检查

语义处理的核心是类型推导。Val 内部类通过 isBool 字段标记子表达式的类型。各类运算符的类型检查规则如下：

- **算术运算符**（+ - * / ^）：两个操作数必须均为数值类型，结果为数值类型。
- **关系运算符**（= <> < <= > >=）：两个操作数必须均为数值类型，结果为布尔类型。
- **逻辑运算符**（& |）：两个操作数必须均为布尔类型，结果为布尔类型。
- **一元逻辑非**（!）：操作数必须为布尔类型，结果为布尔类型。
- **一元取负**（-）：操作数必须为数值类型，结果为数值类型。
- **三元运算符**（?:）：条件必须为布尔类型，真/假分支均为数值类型，结果为数值类型。
- **预定义函数**：所有参数必须为数值类型，结果为数值类型。

特别地，根据规格说明第 2.4.5 节的规定，**布尔类型的值不可作为计算器最后的求值结果**。因此在 calculate() 方法末尾，若最终结果 isBool 为 true，则抛

出 TypeMismatchedException。

3.4.4 预定义函数的处理

预定义函数 sin 和 cos 要求有且仅有 1 个参数；max 和 min 支持可变数目的参数，但至少需要 2 个参数。参数个数的校验规则如下：

- sin/cos：参数个数小于 1 → MissingOperandException；参数个数大于 1 → FunctionCallException。
- max/min：参数个数小于 2 → MissingOperandException。

函数调用通过 handleFunc() 方法实现。进入函数时，LPAREN 的 OpEntry 记录了 valStack 的当前大小作为参数基址；遇到逗号时通过 evalUntil(LPAREN) 归约当前参数；遇到右括号时弹出参数并完成函数计算。

3.4.5 三元运算符的处理

三元运算符 "?:" 因其分为 "?" 和 ":" 两个独立的 Token，需要特殊的语法分析处理。本实现使用三个辅助结构与三个步骤：

步骤一（遇到 "?"）：因为 "?:" 优先级最低，先通过 evalBeforePush 归约条件表达式中所有更高优先级的运算，弹出条件值并暂存于 ternStack，然后将 "?" 压入 opStack 作为标记。

步骤二（遇到 ":"）：通过 evalUntil(QUEST) 归约 "?" 上方（真值分支）的所有运算。弹出真值，从 ternStack 取出对应条件，将条件与真值一并保存后，将 ":" 压入 opStack 作为待完成的标记。

步骤三（":" 归约或表达式结束）：当 ":" 被归约时（可能因遇到更外层的 ":" 或表达式末尾），弹出假值分支的结果，从 ternStack 取出条件与真值，完成三目选择计算并将结果压入 valStack。

这一设计正确处理了三元运算符的右结合嵌套。对于 "$a\,?\,b\,?\,c\,:\,d\,:\,e$"，内层的 "$b\,?\,c\,:\,d$" 会先在遇到第二个 ":" 时完成归约，得到内层结果后再与外层的条件和假值分支组合，最终等价于 $a\,?\,(b\,?\,c\,:\,d)\,:\,e$。

3.5 自定义测试用例设计与覆盖

本实验在完成 standard.xml 全部 16 个测试用例的基础上，额外编写了 49 个自定义扩展测试用例（mytest.xml），系统性地覆盖了规格说明中的边缘情况。测试用例分类如下：

- **右结合性验证**：幂运算（2^{4^3}）、三元运算符（true ? 5 : false ? 1 : 0）。
- **一元取负**：2.5 + (−2.4)、5 − (−2.5)、-3^2（取负优先级高于幂）。
- **三元边界**：多余 ":"（TrinaryOperationException）、缺少真/假值分支（MissingOperandException）、数值条件（TypeMismatchedException）、布尔分支（TypeMismatchedException）、括号/三元不匹配等。
- **布尔终值禁止**：`true`、`false`、比较运算结果、逻辑运算结果作为最终返回值均触发 TypeMismatchedException。
- **函数调用边界**：参数不足（MissingOperandException）、参数过多（FunctionCallException）、逗号边缘情况（cos(3.14,)、max(, 1.8)、max(17, , 87) 等）。
- **非法数值常量**：缺少小数部分（3.e3）、双重小数点（3.3e3.3）、指数中带小数点（3.3E.3）、缺少整数部分（.11）、数值后接字母（12.3Emax）。
- **非法标识符/符号**：未定义标识符（`v4`、`mix`）、非法符号（`@`）。
- **函数调用缺少括号**：max5, 6, 8)、sin5, 3) 触发 FunctionCallException。
- **缺少运算符/运算量**：两个括号表达式连缀、函数调用后紧跟数值。

3.6 异常处理

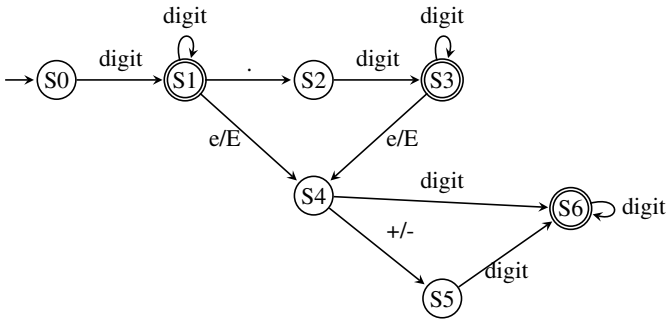
本实验严格按照实验软装置预定义的异常体系进行错误报告：在词法分析、语法分析与语义处理三个阶段一旦发现错误，立即抛出最具体的异常类型，避免使用过于宽泛的父类异常。词法分析阶段负责捕获非法数值、非法标识符与非法符号；语法分析阶段负责检测运算符/运算量缺失、括号不匹配、函数调用形式错误及三元运算符不匹配；语义处理阶段负责类型不匹配与除数为零等运行时语义错误。

异常类型	类别	触发条件（典型示例）
ExpressionException	总基类	表达式错误基类；实现中不直接抛出，仅用于统一捕获。
LexicalException	词法错误基类	词法错误的父类；实现中不直接抛出，仅用于分类。
IllegalDecimalException	词法错误	非法数值常量（如 .11、3.e3、7e、12.3Emax）。
IllegalIdentifierException	词法错误	非法标识符（如 mix、v4）。
IllegalSymbolException	词法错误	非法符号（如 @）。
SyntacticException	语法错误基类	语法错误的父类；实现中不直接抛出，仅用于分类。
MissingOperatorException	语法错误	运算符缺失（如 2 3、)(、函数调用后直接跟数值）。
MissingOperandException	语法错误	运算量缺失（如 2+、max(,1)、cos(3.14,)）。
MissingLeftParenthesisException	语法错误	缺少左括号（如 1+2)）。
MissingRightParenthesisException	语法错误	缺少右括号（如 (1+2）。
FunctionCallException	语法错误	函数调用形式错误或参数个数过多（如 sin(1,2)、max 后直接跟数值）。
TrinaryOperationException	语法错误	三元运算符不匹配（如多余 “:“ 或缺失 “:“）。
EmptyExpressionException	语法错误	输入表达式为空或仅含空白符。
SemanticException	语义错误基类	语义错误的父类；实现中不直接抛出，仅用于分类。
DividedByZeroException	语义错误	除数为 0（如 3/0 或 1/(2-2)）。
TypeMismatchedException	语义错误	类型不匹配（布尔参与算术/数值参与逻辑、三元条件非布尔、布尔终值作为最终结果）。

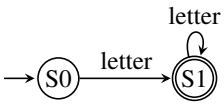
表 3.2 异常处理总结表

3.7 有限状态自动机图表

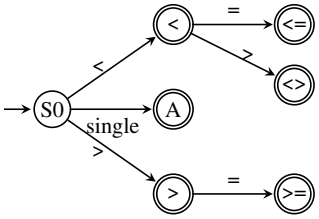
为满足实验要求，词法分析阶段以有限状态自动机（FSA）作为程序蓝图。图 3.1 分别给出了数值常量、标识符/关键字与运算符三类单词的识别自动机，其中双圆表示接受状态。运算符的单字符集合为 `+ - * / ^ & | ! ( ) , ? : =`，双字符运算符由 “<” 或 “>” 前缀扩展得到（<=、>=、<>）。



(a) 数值常量



(b) 标识符/关键字



(c) 运算符与界符

图 3.1 词法分析有限状态自动机示意

4 实验结果与分析

将 `Calculator.java` 编译后，通过实验软装置的自动回归测试工具对全部三套测试用例进行了完整测试，并使用 GUI 界面进行了手动 GUI 界面功能和计算功能测试。测试结果报告如下：

- 编译成功：`Calculator.java` 成功编译，无编译错误或警告。
- GUI 界面基本功能测试：所有 GUI 的输入框、复制粘贴等各个按钮、帮助信息显示等均正常工作。
- 手动 GUI 测试：通过 GUI 界面进行的手动测试表明，所有功能均能正常工作，用户体验良好。
- javadoc 文档生成：成功生成完整的 javadoc 文档，包含所有类、方法和字段的注释信息，文档结构清晰，易于导航。
- 自动回归测试：三套测试用例全部通过，具体结果如下：
  - **simple.xml**：8 个测试用例，8 个通过，通过率 100%。
  - **standard.xml**：16 个测试用例，16 个通过，通过率 100%。
  - **mytest.xml**：49 个自定义扩展测试用例，49 个通过，通过率 100%。

4.1 编译结果

```
● *[master][~/Documents/Courses/大三下/编译原理/homework/Lab/lab4/code/ExprEval]$ make clean
  rm -rf bin/parser doc
● *[master][~/Documents/Courses/大三下/编译原理/homework/Lab/lab4/code/ExprEval]$ make
  cd src && javac -d ../bin -classpath ../bin parser/*.java
```

图 4.1 编译 java 脚本结果截图

```
● *[master][~/Documents/Courses/大三下/编译原理/homework/Lab/lab4/ExprEval]$ make doc
  cd src && javac -d ../bin -classpath ../bin parser/*.java
  cd src && javadoc -private -author -version -d ../doc -classpath ../bin -Xdoclint:none parser/*.java 2>&1
  Loading source file parser/Calculator.java...
  Constructing Javadoc information...
  Building index for all the packages and classes...
  Standard Doclet version 25.0.2
  Building tree for all the packages and classes...
  Generating ../doc/parser/Calculator.html...
  Generating ../doc/parser/Calculator.OpEntry.html...
  Generating ../doc/parser/Calculator.TernState.html...
  Generating ../doc/parser/Calculator.Token.html...
  Generating ../doc/parser/Calculator.TokenType.html...
  Generating ../doc/parser/Calculator.Val.html...
  Generating ../doc/parser/package-summary.html...
  Generating ../doc/parser/package-tree.html...
  Generating ../doc/overview-tree.html...
  Generating ../doc/allclasses-index.html...
  Building index for all classes...
  Generating ../doc/allpackages-index.html...
  Generating ../doc/index-all.html...
  Generating ../doc/search.html...
  Generating ../doc/index.html...
  Generating ../doc/help-doc.html...
```

图 4.2 javadoc 文档生成截图

4.2 GUI 界面基本功能

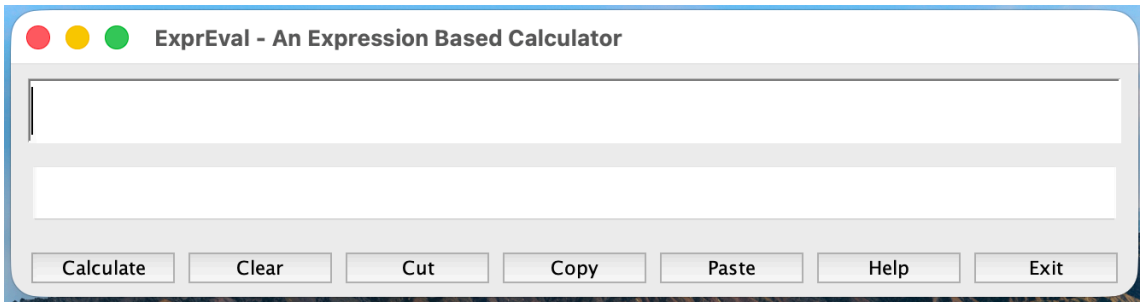


图 4.3 GUI 界面展示

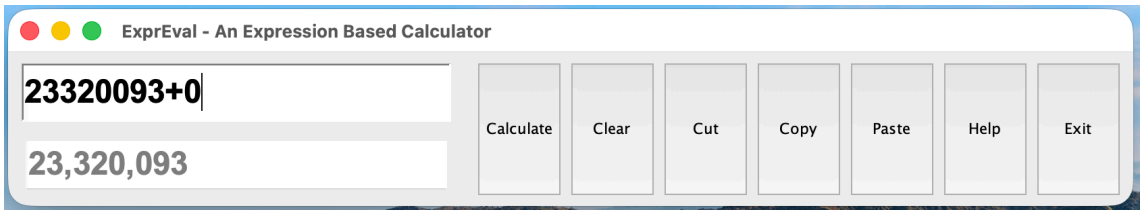


图 4.4 GUI 按钮移动到右方

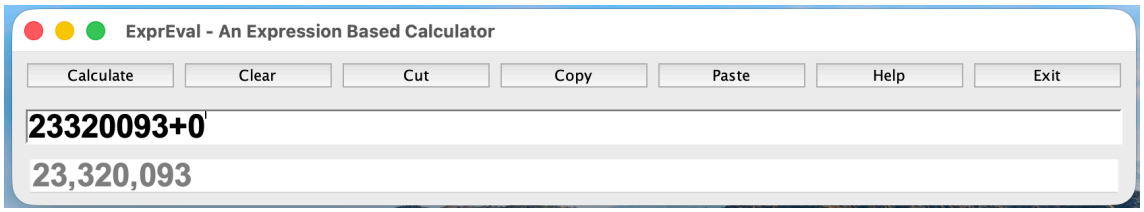


图 4.5 GUI 按钮移动到上方

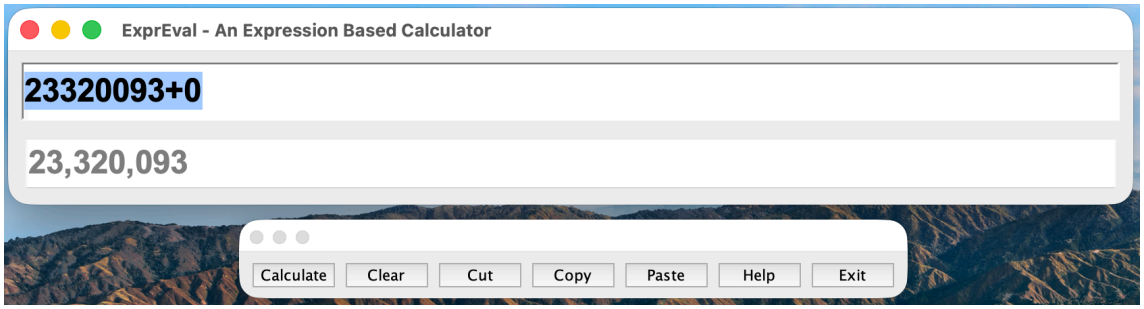


图 4.6 GUI 按钮移动到任意位置

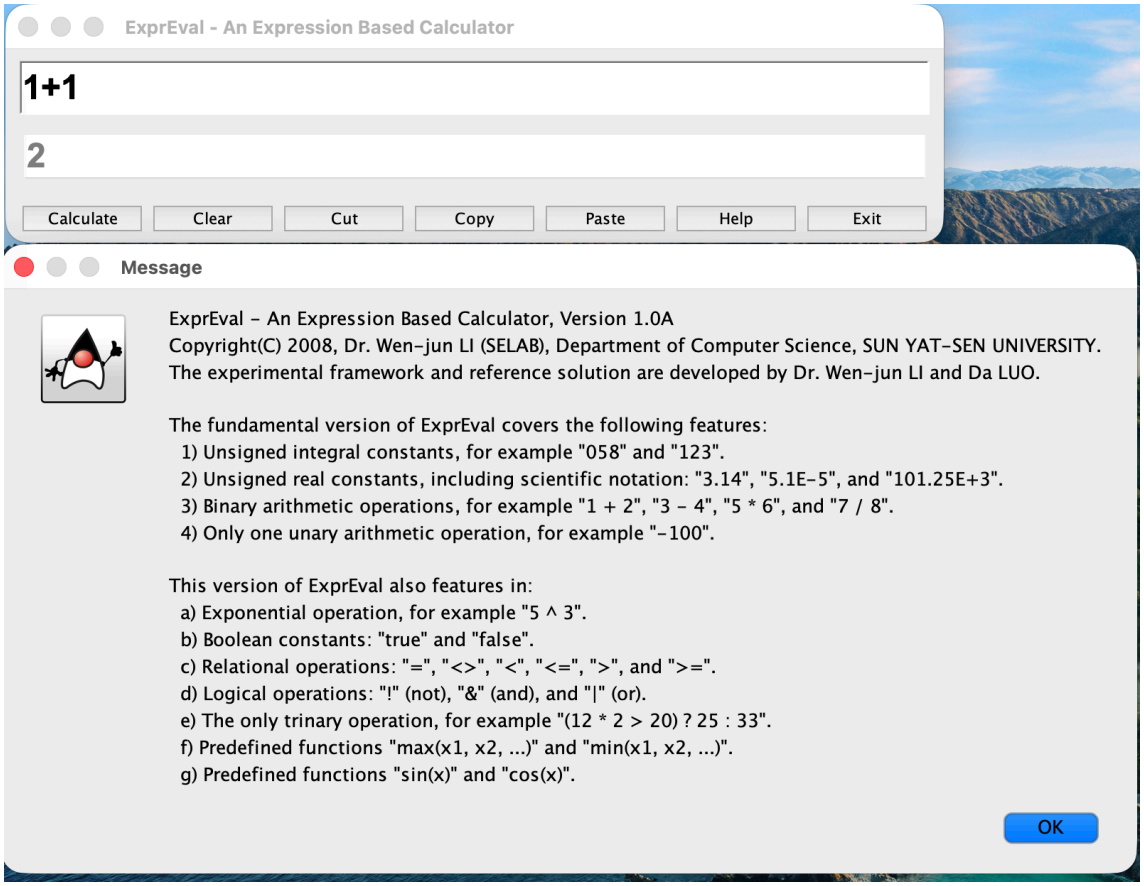


图 4.7 GUI 帮助信息

4.3 自动回归测试结果分析

```
*[master][~/Documents/Courses/大三下/编译原理/homework/Lab/lab4/ExprEval]$ make test-simple
cd src && javac -d ../bin -classpath ../bin parser/*.java
cd bin && java test.ExprEvalTest ../testcases/simple.xml
Testing Procedure
=========================================================

C001 A simple expression.
Input: 9 - 3 * 2
Expected output: 3
Passed !

C002 Expression with arithmetic operations.
Input: 2.25E+2 - (55.5 + 4 * (10 / 2) ^ 2)
Expected output: 69.5
Passed !

C003 Expression with arithmetic operations.
Input: 65 / 5 - 130e-1
Expected output: 0.0
Passed !

E001 Left parenthesis expected.
Input: (2 + 3) ^ 3) - ((1 + 1)
Expected output: MissingLeftParenthesisException
Passed !

E002 Right parenthesis expected.
Input: ((2 + 3) ^ ((3 - 1) + 1)
Expected output: MissingRightParenthesisException
Passed !

E003 Operator expected.
Input: (1 + 2) ^ (3 - 4)  5
Expected output: MissingOperatorException
Passed !

E004 Operand expected.
Input: (1 + 2) ^ (3 - ) + 5
Expected output: MissingOperandException
Passed !

E005 Divided by 0.
Input: 4 / (12 - 3 * 4) + 1
Expected output: DividedByZeroException
Passed !

---------------------------------------------------------
Statistics Report (8 test cases):

        Passed case(s): 8 (100.0%)
        Warning case(s): 0 (0.0%)
        Failed case(s): 0 (0.0%)
=========================================================
```

图 4.8 简单测试用例（simple.xml）运行结果截图

```
*[master][~/Documents/Courses/大三下/编译原理/homework/Lab/lab4/ExprEval]$ make test-standard
Input: sin(min(12, 3 * 5, 2 + 3 ^ 2, 3.14E2))
Expected output: -0.99999
Passed !

E001 Left parenthesis expected.
Input: (2 + 3) ^ 3) - ((1 + 1)
Expected output: MissingLeftParenthesisException
Passed !

E002 Right parenthesis expected.
Input: ((2 + 3) ^ ((3 - 1) + 1)
Expected output: MissingRightParenthesisException
Passed !

E003 Operator expected.
Input: (1 + 2) ^ (3 - 4)  5
Expected output: MissingOperatorException
Passed !

E004 Operand expected.
Input: (1 + 2) ^ (3 - ) + 5
Expected output: MissingOperandException
Passed !

E005 Divided by 0.
Input: 4 / (12 - 3 * 4) + 1
Expected output: DividedByZeroException
Passed !

E006 Type mismatched.
Input: (13 < 2 * 5) + 12
Expected output: TypeMismatchedException
Passed !

E007 Scientific Notation Error.
Input: 4 + 10.E+5 + 1
Expected output: IllegalDecimalException
Passed !

E008 Not a predefined identifier.
Input: 4 + mix(5, 2) + 1
Expected output: IllegalIdentifierException
Passed !

E009 Function call error.
Input: sin(2, 1)
Expected output: FunctionCallException
Passed !

E010 Function call error.
Input: min(2.5)
Expected output: MissingOperandException
Passed !

------------------------------------------------------
Statistics Report (16 test cases):

        Passed case(s): 16 (100.0%)
        Warning case(s): 0 (0.0%)
        Failed case(s): 0 (0.0%)
======================================================
```

图 4.9 标准测试用例（standard.xml）运行结果截图

```
*[master][~/Documents/Courses/大三下/编译原理/homework/Lab/lab4/ExprEval]$ make test-my
Input: 3.3e3.3 + 1
Expected output: IllegalDecimalException
Passed !

E132 Illegal decimal: decimal point in exponent.
Input: 1 + 3.3E.3 + 2
Expected output: IllegalDecimalException
Passed !

E133 Illegal decimal: trailing decimal point.
Input: min(4., 7)
Expected output: IllegalDecimalException
Passed !

E134 Illegal decimal: missing integer part.
Input: .11 + 2
Expected output: IllegalDecimalException
Passed !

E135 Illegal decimal: letter after exponent.
Input: 12.3Emax(4, 5, 6)
Expected output: IllegalDecimalException
Passed !

E136 Illegal identifier in expression.
Input: 5 / v4 + 1
Expected output: IllegalIdentifierException
Passed !

E137 Illegal symbol (@).
Input: (5 @ 4) ? 7 : 8
Expected output: IllegalSymbolException
Passed !

E138 Function call missing left parenthesis.
Input: max5, 6, 8)
Expected output: FunctionCallException
Passed !

E139 Function call missing left parenthesis (sin).
Input: sin5, 3)
Expected output: FunctionCallException
Passed !

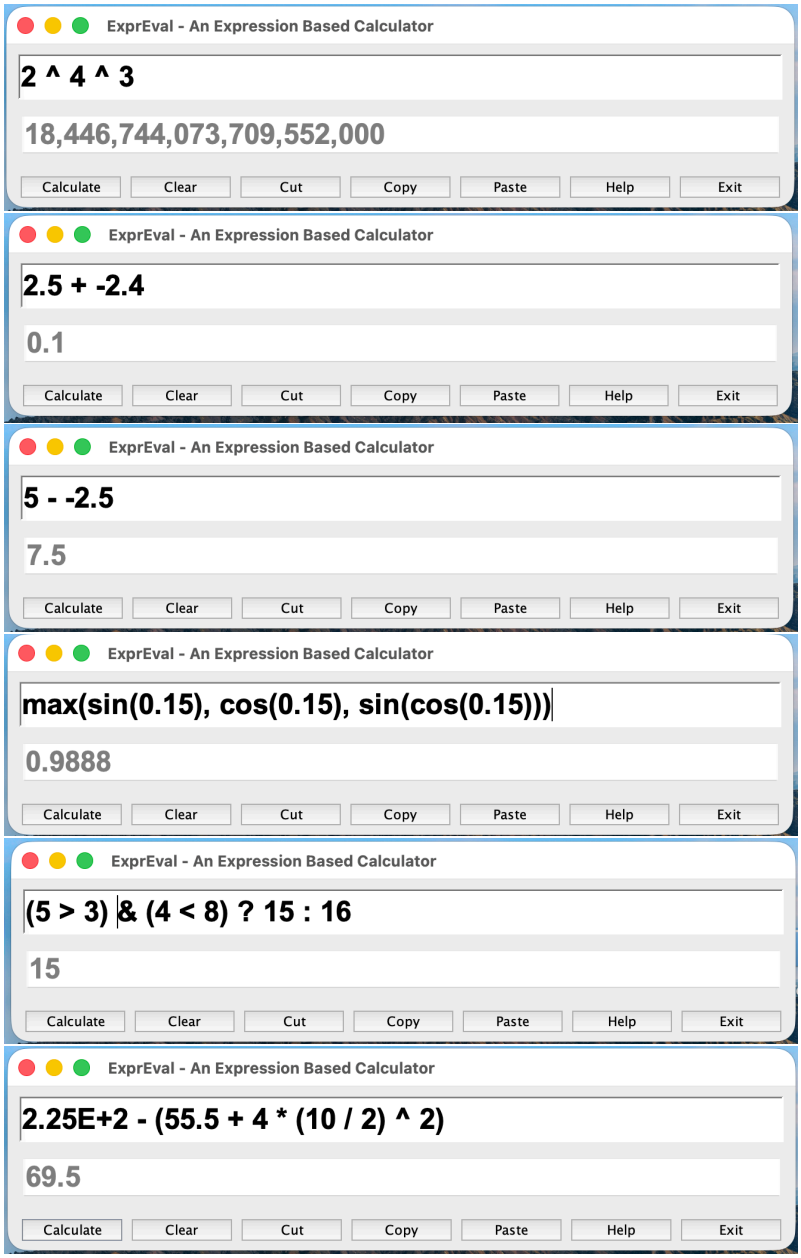
------------------------------------------------------------
Statistics Report (49 test cases):

        Passed case(s): 49 (100.0%)
        Warning case(s): 0 (0.0%)
        Failed case(s): 0 (0.0%)
============================================================
```

图 4.10 自定义扩展测试用例（mytest.xml）运行结果截图

4.4 手动计算测试

由于例子实在是太多了，这里仅展示部分测试用例的计算结果截图。



5 自我评分参照

参照实验规定的记分体系，完成细项列举见表 5.1。

评分项目	**权重**	**对应章节**
0. 总分	100%	-
1. 基本功能需求	55%	第 4 章
1.1 可运行的程序	5%	4.2 节
1.2 实验软装置全面测试	50%	4.1 节
2. 编程风格	5%	第 3 章
2.1 标识符命名	1%	代码
2.2 文档化注释	3%	4.4 节
2.3 程序版面及其他	1%	代码
3. 面向对象建模与设计	15%	第 3 章
3.1 对象属性及其类型设计	7%	3.3–3.4 节
3.2 对象操作及其接口设计	5%	3.4 节
3.3 访问控制设计及其他	3%	3.4 节
4. 实验报告	25%	整篇报告
4.1 二义性分析	2%	3.1 节
4.2 词法分析	3%	3.2 节
4.3 算符优先关系表	5%	3.3 节
4.4 语法分析与语义处理	5%	3.4 节
4.5 实验心得	3%	第 5 章
4.6 文档与源程序组织	2%	第 2 章
4.7 自定义测试用例	5%	3.5 节

表 5.1 评分项目与对应报告章节

6 实验总结

6.1 技术总结

本次实验成功地设计并实现了一个完整的基于表达式的计算器 ExprEval。在技术层面，取得了以下核心成果：

1) **完整的词法分析器**：实现了对 Pascal 风格数值常量（含科学记数法）的精确识别，正确处理了各种非法数值形式（如.11、3.e3、12.3Emax 等）的错误检测，并实现了对布尔常量、预定义函数名的大小写无关识别以及双字符运算符的前瞻匹配。
2) **算符优先语法分析器**：构造了覆盖 11 级优先级的算符优先关系表，正确处理了：
   - 一元取负（级别 9）与二元减法（级别 6）的重载消歧义；
   - 幂运算（级别 8）和三元运算（级别 1）的右结合性质；
   - 预定义函数的可变参数处理与参数个数校验；
   - 三元运算符的分步归约（? 条件暂存 → : 真值暂存 → 三元归约）。
3) **语义处理程序**：实现了严格的类型推导与兼容性检查。通过 Val 内部类的 isBool 字段标记每个子表达式的类型，确保布尔类型值不会参与算术运算、数值类型值不会被误用于逻辑运算，并正确执行了布尔值不得作为最终计算结果的规定。
4) **软件工程实践**：实验结果以 100% 正确率通过了已有的测试用例，并且另外编写了 49 个自定义扩展测试用例覆盖规格说明的各类边缘情况，生成 javadoc 文档化注释，编写了跨平台 Makefile 以支持编译、测试、运行和文档生成的自动化。

6.2 心得体会

这次实验让我从零开始完整地经历了编译器前端的设计与实现全过程。在课堂上学习算符优先分析时，我对优先级表、归约动作这些概念停留在理论层面，而亲手实现一个能处理 11 级优先级、涵盖算术/关系/逻辑/三元运算、具有严格类型检查的表达求值器，让我对这一技术有了深刻的理解。

在实现过程中，最有挑战性的部分是三元运算符 “?:” 的处理。它分为两个独

立的 Token，且处于最低优先级，需要配合辅助栈保存中间状态。调试右结合嵌套时的归约顺序（如 $a\,?\,b\,?\,c\,:\,d\,:\,e$）让我切身体会到算符优先分析方法的核心思想：通过优先级决定归约时机，通过结合性决定同级运算符的归约方向。

另外，重载运算符 – 的处理也值得深思。一元取负与二元减法的消歧义完全依赖于语法上下文（当前期望运算量还是运算符），这让我理解了词法分析与语法分析之间的协作关系——词法分析器只负责切分单词，而不判断单词的语法角色，后者交由语法分析器的上下文来完成。

通过这次实验，我对编译原理中词法分析、语法分析和语义处理三个阶段的工程实现有了系统的认识，也对面向对象设计中的封装（将 Lexer、Parser、Evaluator 融合在 Calculator 类中，通过 private 内部类实现信息隐藏）有了更具体的实践体验。

参考文献

[1] JUN LI W. Lab04-personaltax[Z]. 2026.

[2] JUN LI W. Labeval04-personaltax[Z]. 2026.

附录 A 附录代码

A.1 Calculator.java 代码

Listing A.1 Calculator.java

```
/**
 * @Copyright(C) 2008 Software Engineering Laboratory (SELAB), Department of
   Computer
 * Science, SUN YAT-SEN UNIVERSITY. All rights reserved.
**/

package parser;

import exceptions.*;
import java.util.*;

/**
 * Main program of the expression based calculator ExprEval.
 * Implements a complete lexer, operator-precedence parser, and semantic
 * evaluator for arithmetic/boolean expressions.
 *
 * @author [PENDING your name]
 * @version 1.00 (Last update: [PENDING the last update])
**/
public class Calculator
{
    //   Token types
    private enum TokenType {
        NUMBER, BOOL, FUNC,
        PLUS, MINUS, MUL, DIV, POWER,
        EQ, NE, LT, LE, GT, GE,
        AND, OR, NOT,
        QUEST, COLON, COMMA,
        LPAREN, RPAREN,
        END
```

```
30      }
31
32      //   Token
33      private static class Token {
34          TokenType type;
35          String   text;
36          double   numVal;
37          boolean  boolVal;
38          String   funcName;
39
40          Token(TokenType t, String s) { type = t; text = s; }
41      }
42
43      //   Value with type tag
44      private static class Val {
45          double  v;
46          boolean isBool;
47          Val(double vv, boolean b) { v = vv; isBool = b; }
48      }
49
50      //   Operator stack entry
51      private static class OpEntry {
52          Token   tok;
53          boolean isUnary;
54          int     valBase;   // valStack size when FUNC's '(' was pushed
55          OpEntry(Token t, boolean u) { tok = t; isUnary = u; valBase = -1; }
56          OpEntry(Token t, boolean u, int vb) {
57              tok = t; isUnary = u; valBase = vb;
58          }
59      }
60
61      //   Ternary state
62      private static class TernState {
63          Val cond;
64          Val trueBranch;
65          TernState(Val c) { cond = c; }
66      }
67
```

```
68      //   Precedence (higher = tighter)
69      // Level 11 (tightest): ( )   — handled specially
70      // Level 10: sin cos max min — handled specially
71      // Level  9: unary -
72      // Level  8: ^
73      // Level  7: * /
74      // Level  6: + -
75      // Level  5: = <> < <= > >=
76      // Level  4: !
77      // Level  3: &
78      // Level  2: |
79      // Level  1 (loosest): ?:
80      private static int prec(TokenType t, boolean unary) {
81          if (unary && t == TokenType.MINUS) return 9;
82          switch (t) {
83              case QUEST: case COLON: return 1;
84              case OR:                return 2;
85              case AND:               return 3;
86              case NOT:               return 4;
87              case EQ: case NE:
88              case LT: case LE:
89              case GT: case GE:       return 5;
90              case PLUS: case MINUS: return 6;
91              case MUL: case DIV:     return 7;
92              case POWER:             return 8;
93              default:                return 0;
94          }
95      }
96
97      private static boolean rightAssoc(TokenType t) {
98          return t == TokenType.POWER || t == TokenType.QUEST;
99      }
100
101     //
102     //  LEXER
103     //
104
105     private List<Token> lex(String expr) throws LexicalException {
```

```
106         List<Token> tokens = new ArrayList<>();
107         int i = 0, n = expr.length();
108         while (i < n) {
109             char c = expr.charAt(i);
110             if (c == ' ') { i++; continue; }
111 
112             if (c >= '0' && c <= '9' || c == '.')
113                 i = lexNumber(expr, i, tokens);
114             else if (Character.isLetter(c))
115                 i = lexIdent(expr, i, tokens);
116             else
117                 i = lexSymbol(expr, i, tokens);
118         }
119         tokens.add(new Token(TokenType.END, ""));
120         return tokens;
121     }
122 
123     private int lexNumber(String s, int i, List<Token> out)
124             throws IllegalDecimalException {
125         int start = i, n = s.length();
126 
127         // Number starting with '.' is invalid (e.g. ".11" — missing integer
     part)
128         if (s.charAt(i) == '.')
129             throw new IllegalDecimalException();
130 
131         while (i < n && Character.isDigit(s.charAt(i))) i++;
132 
133         if (i < n && s.charAt(i) == '.') {
134             i++;
135             if (i >= n || !Character.isDigit(s.charAt(i)))
136                 throw new IllegalDecimalException();
137             while (i < n && Character.isDigit(s.charAt(i))) i++;
138         }
139 
140         if (i < n && (s.charAt(i) == 'e' || s.charAt(i) == 'E')) {
141             i++;
142             if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-')) i++;
```

```
143             if (i >= n || !Character.isDigit(s.charAt(i)))
144                 throw new IllegalDecimalException();
145             while (i < n && Character.isDigit(s.charAt(i))) i++;
146         }
147 
148         // Trailing garbage: if next char is '.' or letter, the decimal is
    malformed
149         // e.g. "3.3e3.3" or "12.3Emax" or "4.E+3"
150         if (i < n && (s.charAt(i) == '.' || Character.isLetter(s.charAt(i))))
151             throw new IllegalDecimalException();
152 
153         Token tok = new Token(TokenType.NUMBER, s.substring(start, i));
154         tok.numVal = Double.parseDouble(tok.text);
155         out.add(tok);
156         return i;
157     }
158 
159     private int lexIdent(String s, int i, List<Token> out)
160             throws IllegalIdentifierException {
161         int start = i, n = s.length();
162         while (i < n && Character.isLetter(s.charAt(i))) i++;
163         String word = s.substring(start, i);
164         String lo = word.toLowerCase();
165 
166         if (lo.equals("true")) {
167             Token tok = new Token(TokenType.BOOL, word);
168             tok.boolVal = true; tok.numVal = 1.0; out.add(tok);
169         } else if (lo.equals("false")) {
170             Token tok = new Token(TokenType.BOOL, word);
171             tok.boolVal = false; tok.numVal = 0.0; out.add(tok);
172         } else if (lo.equals("sin") || lo.equals("cos") ||
173                    lo.equals("max") || lo.equals("min")) {
174             Token tok = new Token(TokenType.FUNC, word);
175             tok.funcName = lo; out.add(tok);
176         } else {
177             throw new IllegalIdentifierException();
178         }
179         return i;
```

```
180     }
181
182     private int lexSymbol(String s, int i, List<Token> out)
183             throws IllegalSymbolException {
184         char c = s.charAt(i);
185         switch (c) {
186         case '+': out.add(new Token(TokenType.PLUS,  "+")); i++; break;
187         case '*': out.add(new Token(TokenType.MUL,   "*")); i++; break;
188         case '/': out.add(new Token(TokenType.DIV,   "/")); i++; break;
189         case '^': out.add(new Token(TokenType.POWER, "^")); i++; break;
190         case '&': out.add(new Token(TokenType.AND,   "&")); i++; break;
191         case '|': out.add(new Token(TokenType.OR,    "|")); i++; break;
192         case '!': out.add(new Token(TokenType.NOT,   "!")); i++; break;
193         case '(': out.add(new Token(TokenType.LPAREN,"(")); i++; break;
194         case ')': out.add(new Token(TokenType.RPAREN,")")); i++; break;
195         case ',': out.add(new Token(TokenType.COMMA, ",")); i++; break;
196         case '?': out.add(new Token(TokenType.QUEST, "?")); i++; break;
197         case ':': out.add(new Token(TokenType.COLON, ":")); i++; break;
198         case '-': out.add(new Token(TokenType.MINUS, "-")); i++; break;
199         case '=': out.add(new Token(TokenType.EQ,    "=")); i++; break;
200         case '<':
201             i++;
202             if (i < s.length() && s.charAt(i) == '=') {
203                 out.add(new Token(TokenType.LE, "<=")); i++;
204             } else if (i < s.length() && s.charAt(i) == '>') {
205                 out.add(new Token(TokenType.NE, "<>")); i++;
206             } else {
207                 out.add(new Token(TokenType.LT, "<"));
208             }
209             break;
210         case '>':
211             i++;
212             if (i < s.length() && s.charAt(i) == '=') {
213                 out.add(new Token(TokenType.GE, ">=")); i++;
214             } else {
215                 out.add(new Token(TokenType.GT, ">"));
216             }
217             break;
```

```
218         default:
219             throw new IllegalSymbolException();
220         }
221         return i;
222     }
223 
224     //
225     //  PARSER & EVALUATOR
226     //
227 
228     private List<Token>      tokens;
229     private int              pos;
230     private Stack<Val>       valStack;
231     private Stack<OpEntry>   opStack;
232     private Stack<TernState> ternStack;
233 
234     public double calculate(String expression) throws ExpressionException {
235         tokens    = lex(expression);
236         pos       = 0;
237         valStack  = new Stack<>();
238         opStack   = new Stack<>();
239         ternStack = new Stack<>();
240 
241         boolean expectOpnd = true;
242         boolean sawComma   = false;
243 
244         while (pos < tokens.size()) {
245             Token t = tokens.get(pos);
246 
247             switch (t.type) {
248 
249             case NUMBER:
250                 if (!expectOpnd) throw new MissingOperatorException();
251                 valStack.push(new Val(t.numVal, false));
252                 expectOpnd = false;
253                 sawComma = false;
254                 pos++;
255                 break;
```

```
256
257             case BOOL:
258                 if (!expectOpnd) throw new MissingOperatorException();
259                 valStack.push(new Val(t.numVal, true));
260                 expectOpnd = false;
261                 sawComma = false;
262                 pos++;
263                 break;
264
265             case FUNC:
266                 if (!expectOpnd) throw new MissingOperatorException();
267                 opStack.push(new OpEntry(t, false));
268                 expectOpnd = true;
269                 sawComma = false;
270                 pos++;
271                 if (pos >= tokens.size()
272                         || tokens.get(pos).type != TokenType.LPAREN)
273                     throw new FunctionCallException();
274                 break;
275
276             case LPAREN:
277                 if (!expectOpnd) throw new MissingOperatorException();
278                 if (!opStack.isEmpty()
279                         && opStack.peek().tok.type == TokenType.FUNC)
280                     opStack.push(new OpEntry(t, false, valStack.size()));
281                 else
282                     opStack.push(new OpEntry(t, false));
283                 expectOpnd = true;
284                 sawComma = false;
285                 pos++;
286                 break;
287
288             case RPAREN:
289                 if (expectOpnd) {
290                     // If a comma was just seen, the comma expected an
    operand
291                     if (sawComma) throw new MissingOperandException();
292                     if (!opStack.isEmpty()
```

```
293                         && opStack.peek().tok.type == TokenType.LPAREN) {
294                     OpEntry lp = opStack.pop();
295                     if (!opStack.isEmpty()
296                             && opStack.peek().tok.type == TokenType.FUNC)
    {
297                         OpEntry fe = opStack.pop();
298                         handleFunc(fe.tok, lp.valBase);
299                     } else {
300                         throw new EmptyExpressionException();
301                     }
302                 } else {
303                     throw new MissingOperandException();
304                 }
305             } else {
306                 evalUntil(TokenType.LPAREN);
307                 if (opStack.isEmpty()
308                         || opStack.peek().tok.type != TokenType.LPAREN)
309                     throw new MissingLeftParenthesisException();
310                 OpEntry lp = opStack.pop();
311                 if (!opStack.isEmpty()
312                         && opStack.peek().tok.type == TokenType.FUNC) {
313                     OpEntry fe = opStack.pop();
314                     handleFunc(fe.tok, lp.valBase);
315                 }
316             }
317             expectOpnd = false;
318             sawComma = false;
319             pos++;
320             break;
321
322         case COMMA:
323             if (expectOpnd) throw new MissingOperandException();
324             evalUntil(TokenType.LPAREN);
325             if (opStack.isEmpty()
326                     || opStack.peek().tok.type != TokenType.LPAREN)
327                 throw new MissingLeftParenthesisException();
328             expectOpnd = true;
329             sawComma = true;
```

```
330                 pos++;
331                 break;
332
333             case NOT:
334                 if (!expectOpnd) throw new MissingOperatorException();
335                 opStack.push(new OpEntry(t, false));
336                 expectOpnd = true;
337                 sawComma = false;
338                 pos++;
339                 break;
340
341             case MINUS:
342                 if (expectOpnd) {
343                     opStack.push(new OpEntry(t, true));
344                     sawComma = false;
345                 } else {
346                     evalBeforePush(t, false);
347                     opStack.push(new OpEntry(t, false));
348                     expectOpnd = true;
349                 }
350                 pos++;
351                 break;
352
353             case PLUS: case MUL: case DIV: case POWER:
354             case EQ: case NE: case LT: case LE: case GT: case GE:
355             case AND: case OR:
356                 if (expectOpnd) throw new MissingOperandException();
357                 evalBeforePush(t, false);
358                 opStack.push(new OpEntry(t, false));
359                 expectOpnd = true;
360                 pos++;
361                 break;
362
363             case QUEST:
364                 if (expectOpnd) throw new MissingOperandException();
365                 evalBeforePush(t, false);
366                 {
367                     Val c = valStack.pop();
```

```
368                     ternStack.push(new TernState(c));
369                 }
370                 opStack.push(new OpEntry(t, false));
371                 expectOpnd = true;
372                 pos++;
373                 break;
374 
375             case COLON:
376                 if (expectOpnd) throw new MissingOperandException();
377                 evalUntil(TokenType.QUEST);
378                 if (opStack.isEmpty()
379                         || opStack.peek().tok.type != TokenType.QUEST)
380                     throw new TrinaryOperationException();
381                 opStack.pop();
382                 {
383                     Val tv = valStack.pop();
384                     TernState ts = ternStack.pop();
385                     ts.trueBranch = tv;
386                     ternStack.push(ts);
387                 }
388                 opStack.push(new OpEntry(t, false));
389                 expectOpnd = true;
390                 pos++;
391                 break;
392 
393             case END:
394                 if (expectOpnd) {
395                     if (valStack.isEmpty() && opStack.isEmpty())
396                         throw new EmptyExpressionException();
397                     throw new MissingOperandException();
398                 }
399                 while (!opStack.isEmpty()) {
400                     OpEntry oe = opStack.pop();
401                     if (oe.tok.type == TokenType.LPAREN)
402                         throw new MissingRightParenthesisException();
403                     evalOp(oe);
404                 }
405                 pos++;
```

```
406                 break;
407
408             default:
409                 pos++;
410             }
411         }
412
413         if (valStack.size() != 1)
414             throw new MissingOperatorException();
415
416         Val result = valStack.pop();
417         if (result.isBool)
418             throw new TypeMismatchedException();
419         return result.v;
420     }
421
422     //   OPP stack helpers
423
424     private void evalUntil(TokenType marker) throws ExpressionException {
425         while (!opStack.isEmpty()) {
426             TokenType top = opStack.peek().tok.type;
427             if (top == marker) break;
428             if (top == TokenType.LPAREN) break;
429             OpEntry oe = opStack.pop();
430             evalOp(oe);
431         }
432     }
433
434     private void evalBeforePush(Token incoming, boolean isUnary)
435             throws ExpressionException {
436         int pIn = prec(incoming.type, isUnary);
437         while (!opStack.isEmpty()) {
438             OpEntry top = opStack.peek();
439             TokenType tt = top.tok.type;
440             if (tt == TokenType.LPAREN || tt == TokenType.QUEST) break;
441             int pTop = prec(tt, top.isUnary);
442             if (rightAssoc(incoming.type)) {
443                 if (pTop <= pIn) break;
```

```
444             } else {
445                 if (pTop < pIn) break;
446             }
447             opStack.pop();
448             evalOp(top);
449         }
450     }
451 
452     //   Semantic actions
453 
454     private void evalOp(OpEntry oe) throws ExpressionException {
455         Token op = oe.tok;
456 
457         // Ternary ':' completion
458         if (op.type == TokenType.COLON) {
459             Val falseVal = valStack.pop();
460             TernState ts = ternStack.pop();
461             if (!ts.cond.isBool) throw new TypeMismatchedException();
462             Val res = (ts.cond.v != 0) ? ts.trueBranch : falseVal;
463             valStack.push(res);
464             return;
465         }
466 
467         // Unary minus
468         if (op.type == TokenType.MINUS && oe.isUnary) {
469             Val a = valStack.pop();
470             if (a.isBool) throw new TypeMismatchedException();
471             valStack.push(new Val(-a.v, false));
472             return;
473         }
474 
475         // Unary NOT
476         if (op.type == TokenType.NOT) {
477             Val a = valStack.pop();
478             if (!a.isBool) throw new TypeMismatchedException();
479             valStack.push(new Val(a.v != 0 ? 0.0 : 1.0, true));
480             return;
481         }
```

```
482
483         // Binary operators
484         Val b = valStack.pop();
485         Val a = valStack.pop();
486
487         switch (op.type) {
488         case PLUS:
489             if (a.isBool || b.isBool) throw new TypeMismatchedException();
490             valStack.push(new Val(a.v + b.v, false));
491             break;
492         case MINUS:
493             if (a.isBool || b.isBool) throw new TypeMismatchedException();
494             valStack.push(new Val(a.v - b.v, false));
495             break;
496         case MUL:
497             if (a.isBool || b.isBool) throw new TypeMismatchedException();
498             valStack.push(new Val(a.v * b.v, false));
499             break;
500         case DIV:
501             if (a.isBool || b.isBool) throw new TypeMismatchedException();
502             if (b.v == 0.0) throw new DividedByZeroException();
503             valStack.push(new Val(a.v / b.v, false));
504             break;
505         case POWER:
506             if (a.isBool || b.isBool) throw new TypeMismatchedException();
507             valStack.push(new Val(Math.pow(a.v, b.v), false));
508             break;
509         case EQ:
510             if (a.isBool || b.isBool) throw new TypeMismatchedException();
511             valStack.push(new Val(a.v == b.v ? 1.0 : 0.0, true));
512             break;
513         case NE:
514             if (a.isBool || b.isBool) throw new TypeMismatchedException();
515             valStack.push(new Val(a.v != b.v ? 1.0 : 0.0, true));
516             break;
517         case LT:
518             if (a.isBool || b.isBool) throw new TypeMismatchedException();
519             valStack.push(new Val(a.v < b.v ? 1.0 : 0.0, true));
```

```
520             break;
521         case LE:
522             if (a.isBool || b.isBool) throw new TypeMismatchedException();
523             valStack.push(new Val(a.v <= b.v ? 1.0 : 0.0, true));
524             break;
525         case GT:
526             if (a.isBool || b.isBool) throw new TypeMismatchedException();
527             valStack.push(new Val(a.v > b.v ? 1.0 : 0.0, true));
528             break;
529         case GE:
530             if (a.isBool || b.isBool) throw new TypeMismatchedException();
531             valStack.push(new Val(a.v >= b.v ? 1.0 : 0.0, true));
532             break;
533         case AND:
534             if (!a.isBool || !b.isBool) throw new TypeMismatchedException();
535             valStack.push(new Val((a.v != 0 && b.v != 0) ? 1.0 : 0.0, true));
536             break;
537         case OR:
538             if (!a.isBool || !b.isBool) throw new TypeMismatchedException();
539             valStack.push(new Val((a.v != 0 || b.v != 0) ? 1.0 : 0.0, true));
540             break;
541         default:
542             break;
543         }
544     }
545
546     private void handleFunc(Token func, int base) throws ExpressionException
    {
547         String name = func.funcName;
548         int nArgs = valStack.size() - base;
549
550         if (name.equals("sin") || name.equals("cos")) {
551             if (nArgs < 1) throw new MissingOperandException();
552             if (nArgs > 1) throw new FunctionCallException();
553             Val a = valStack.pop();
554             if (a.isBool) throw new TypeMismatchedException();
555             double r = name.equals("sin") ? Math.sin(a.v) : Math.cos(a.v);
556             valStack.push(new Val(r, false));
```

```
557         } else { // max or min
558             if (nArgs < 2) throw new MissingOperandException();
559             // Args are on stack in order [..., arg1, arg2, ... argN]
560             // Pop them in reverse
561             double[] args = new double[nArgs];
562             for (int i = nArgs - 1; i >= 0; i--) {
563                 Val a = valStack.pop();
564                 if (a.isBool) throw new TypeMismatchedException();
565                 args[i] = a.v;
566             }
567             double result = args[0];
568             if (name.equals("max")) {
569                 for (int i = 1; i < nArgs; i++)
570                     if (args[i] > result) result = args[i];
571             } else {
572                 for (int i = 1; i < nArgs; i++)
573                     if (args[i] < result) result = args[i];
574             }
575             valStack.push(new Val(result, false));
576         }
577     }
578 
579     //   Embedded tests (run this class directly)
580     private static class EvalCase {
581         String expr;
582         Double expected;
583         Class<? extends ExpressionException> expectedEx;
584         EvalCase(String e, double v) { expr = e; expected = v; }
585         EvalCase(String e, Class<? extends ExpressionException> ex) {
586             expr = e; expectedEx = ex;
587         }
588     }
589 
590     private static boolean approx(double a, double b) {
591         return Math.abs(a - b) <= 1e-4;
592     }
593 
594     private static void runLexSmokeTests(Calculator calc) {
```

```
595         String[] samples = {
596             "12.3E-4",
597             "true",
598             "sin(3.14)",
599             "max(1,2,3)",
600             "1<=2"
601         };
602         System.out.println("== Lexer smoke tests ==");
603         for (String s : samples) {
604             try {
605                 List<Token> toks = calc.lex(s);
606                 StringBuilder sb = new StringBuilder();
607                 for (Token t : toks) {
608                     if (t.type == TokenType.END) continue;
609                     sb.append(t.type).append("(")
610                       .append(t.text).append(") ");
611                 }
612                 System.out.println(s + " => " + sb.toString().trim());
613             } catch (LexicalException e) {
614                 System.out.println(s + " => " + e.getClass().getSimpleName())
    ;
615             }
616         }
617         System.out.println();
618     }
619
620     private static void runEvalTests(Calculator calc) {
621         EvalCase[] cases = new EvalCase[] {
622             new EvalCase("2+3*4", 14.0),
623             new EvalCase("sin(0)", 0.0),
624             new EvalCase("max(3,2,6)", 6.0),
625             new EvalCase("true?5:6", 5.0),
626             new EvalCase("3/0", DividedByZeroException.class),
627             new EvalCase("true", TypeMismatchedException.class),
628             new EvalCase("sin(1,2)", FunctionCallException.class),
629             new EvalCase("4.E+3", IllegalDecimalException.class),
630             new EvalCase("mix", IllegalIdentifierException.class),
631             new EvalCase("5@4", IllegalSymbolException.class),
```

```
632             new EvalCase("1+2)", MissingLeftParenthesisException.class),
633             new EvalCase("(1+2", MissingRightParenthesisException.class),
634             new EvalCase("(1+2)3", MissingOperatorException.class),
635             new EvalCase("1+", MissingOperandException.class),
636             new EvalCase("false?9:true?1:3:5", TrinaryOperationException.
    class),
637             new EvalCase("   ", EmptyExpressionException.class)
638         };
639
640         System.out.println("== Evaluator tests ==");
641         int passed = 0;
642         for (EvalCase c : cases) {
643             try {
644                 double r = calc.calculate(c.expr);
645                 if (c.expectedEx != null) {
646                     System.out.println("[FAIL] " + c.expr
647                         + " => " + r + ", expected "
648                         + c.expectedEx.getSimpleName());
649                     continue;
650                 }
651                 if (!approx(r, c.expected)) {
652                     System.out.println("[FAIL] " + c.expr
653                         + " => " + r + ", expected " + c.expected);
654                     continue;
655                 }
656                 System.out.println("[PASS] " + c.expr + " => " + r);
657                 passed++;
658             } catch (ExpressionException e) {
659                 if (c.expectedEx != null && c.expectedEx.isInstance(e)) {
660                     System.out.println("[PASS] " + c.expr + " => "
661                         + e.getClass().getSimpleName());
662                     passed++;
663                 } else {
664                     System.out.println("[FAIL] " + c.expr + " => "
665                         + e.getClass().getSimpleName());
666                 }
667             }
668         }
```

```
669         System.out.println("Summary: " + passed + "/" + cases.length
670             + " passed");
671     }
672 
673     public static void main(String[] args) {
674         Calculator calc = new Calculator();
675         runLexSmokeTests(calc);
676         runEvalTests(calc);
677     }
678 }
```